Chapter 5

Document Embeddings

5.1 Introduction and Motivation

The thematic relevance scores developed in Chapter 4 provide a structured representation of the relationship between each article and five predefined financial risk categories. They answer a specific question: *how strongly is article related to risk category?*
This representation is necessarily conditional on the categories defined ex ante. It cannot, by itself, capture the complete semantic content of an article or measure whether two articles discuss similar information using different terminology. A second representation is therefore required.

For each article (i), the objective of the present chapter is to construct a document-level semantic vector

$$E_i \in \mathbb{R}^d, \tag{5.1}$$

where d denotes the embedding dimension.
In contrast to thematic scores, the individual coordinates of E_i do not carry a direct interpretation. What matters instead is the relative position of the vectors within the embedding space: articles that are semantically similar are expected to be located close to each other.
This distinction is central to the empirical strategy:

$$S_{i,k,m} \quad \rightarrow \quad \text{theme-specific relevance,} \tag{5.2}$$

$$E_i \quad \rightarrow \quad \text{general semantic representation.} \tag{5.3}$$

Document embeddings are subsequently used to construct measures of semantic novelty and semantic drift.

5.2 Sentence-Transformer Representation

Embedding models convert each article into a numerical vector that represents its semantic content. The main idea is that articles with a similar meaning should also have similar embeddings even if they do not use exactly the same words. Two articles may describe the same event while using different

expressions or terminology.
This is useful in with news data, where the same information is often reported in different ways. Looking only at the words shared by two articles could therefore miss relevant similarities and embeddings provide a way to compare the articles beyond this lexical overlap and focus instead on the meaning conveyed by the text.
The project design compared two Sentence-Transformer methods:

- `sentence-transformers/all-MiniLM-L6-v2`;
- `sentence-transformers/all-mpnet-base-v2`.

Rather than selecting one model a priori, both candidates were benchmarked on actual articles from the corpus.

5.3 Model Selection

5.3.1 MiniLM and MPNet

The two models differ in dimensionality, context capacity, and computational cost.
MiniLM produces

$$E_i \in \mathbb{R}^{384}, \tag{5.4}$$

with a maximum sequence length of 256 tokens.
MPNet produces

$$E_i \in \mathbb{R}^{768}, \tag{5.5}$$

with a maximum sequence length of 384 tokens.
The higher dimensionality of MPNet does not by itself imply a superior representation for the present application. Model selection must instead account for semantic representation, document length, computational cost, and the scale of the corpus.

5.3.2 Real-Corpus Benchmark

The two models were benchmarked using 1,000 real articles from the analytical corpus.
MiniLM achieved an encoding throughput of approximately 172.62 articles per second, whereas MPNet processed approximately 20.45 articles per second. MPNet was therefore approximately 8.4 times slower in the benchmark.
MPNet has a longer context window, but this only slightly reduced the number of articles that exceeded the limit. Approximately 94.4% of articles exceeded MiniLM's 256-token limit, while approximately 89.9% exceeded MPNet's 384-token limit.
Thus, moving from MiniLM to MPNet did not solve the fundamental long-document problem. At the same time, MPNet substantially increased computational cost and doubled the dimensionality of the stored vectors.

MiniLM was therefore selected as the embedding backbone.
This decision should not be interpreted as a general claim that MiniLM is superior to MPNet. It is a corpus-specific engineering and methodological choice based on the trade-off observed in the present application.

5.4 The Long-Document Problem

The news articles in the corpus are substantially longer than the native MiniLM context window.
In the initial document-length benchmark the mean article length was approximately 930 tokens and the 95th percentile exceeded 2,000 tokens.
Consequently, the standard pipeline

$$\text{title + body} \rightarrow \text{MiniLM} \rightarrow \text{automatic truncation} \tag{5.6}$$

would discard a substantial portion of the text for the overwhelming majority of articles.
This would be problematic for the present application. The objective is to obtain a *document-level* semantic representation, while standard truncation would primarily represent the beginning of the article.

5.4.1 Strategy

A chunking strategy was therefore introduced.
For article i, the model input is constructed as

$$X_i = \text{title}_i + \texttt{\textbackslash n\textbackslash n} + \text{body}_i. \tag{5.7}$$

The complete text is tokenized without truncation.
MiniLM supports a maximum sequence length of 256 tokens. Since two positions are reserved for model-specific special tokens, each content chunk contains at most 256-2 = 254 content tokens.
The complete token sequence is therefore partitioned into non-overlapping chunks of 254 content tokens.

5.5 Chunk-Level Embeddings

Suppose article i contains C_i chunks. MiniLM produces one 384-dimensional vector for each chunk:

$$e_{i,c} \in \mathbb{R}^{384}, \qquad c = 1, \ldots, C_i. \tag{5.8}$$

Each chunk embedding is then L_2-normalized:

$$\hat{e}_{i,c} = \frac{e_{i,c}}{|e_{i,c}|_2}. \tag{5.9}$$

This design ensures that each content token contributes once to the document representation. An overlapping strategy would cause tokens appearing in overlap regions to be represented multiple times

and would therefore introduce implicit additional weighting.
Zero overlap is thus a simple and reproducible implementation choice. It should be interpreted as part of the document-embedding specification developed for this thesis rather than as an intrinsic requirement of MiniLM.

5.6 Document-Level Mean Pooling

The normalized chunk embeddings are aggregated through arithmetic mean pooling:

$$\bar{e}_i = \frac{1}{C_i} \sum_{c=1}^{C_i} \hat{e}_{i,c}. \tag{5.10}$$

The resulting vector is then L_2-normalized:

$$E_i = \frac{\bar{e}_i}{\|\bar{e}_i\|_2}. \tag{5.11}$$

Consequently, $\|E_i\|_2 = 1$, and the final output is a single 384-dimensional vector for each article.
Mean pooling provides a method for combining information distributed across the complete article while preserving the native dimensionality of the MiniLM embedding space.
Importantly, the dimensionality of 384 was not independently chosen during the thesis. It is the native output dimension of `all-MiniLM-L6-v2`. Chunking and pooling preserve this dimensionality.

5.7 Cosine Similarity and Normalization

For two normalized article embeddings E_i and E_j, cosine similarity is defined as

$$\cos(E_i, E_j) = \frac{E_i^\top E_j}{\|E_i\|_2 \|E_j\|_2}. \tag{5.12}$$

Since the document embeddings are L_2-normalized, with $\|E_i\|_2 = \|E_j\|_2 = 1$, this simplifies to

$$\cos(E_i, E_j) = E_i^\top E_j, \tag{5.13}$$

and is the cosine similarity.
This property is especially useful for the emerging-risk measures developed in the following chapter, where semantic relationships between current and historical news articles are repeatedly evaluated.
High cosine similarity indicates semantically similar articles, while lower cosine similarity indicates greater semantic distance.

5.8 Validation of the Chunking Strategy

The first full-document benchmark was conducted on 1,000 articles.
The mean number of chunks per article was approximately 4166 and only 5.6% of the sampled articles could be represented using a single chunk.

These results confirmed that the long-document issue was not confined to a small number of extreme observations. Multi-chunk processing was necessary for most of the corpus.

A larger audit was subsequently conducted to verify that the initial benchmark was not driven by a particular period of the sample and similar results were obtained. The temporal audit therefore provided additional evidence that the chunking requirements were stable across the sample period.

5.9 Final Embedding Specification

The final document-embedding specification is identified as

`all-MiniLM-L6-v2_chunk254_mean_l2_v1`,

and its defining characteristics are:

- backbone: `sentence-transformers/all-MiniLM-L6-v2`;
- output dimension: 384;
- input: article title followed by article body;
- maximum model sequence length: 256;
- content tokens per chunk: 254;
- chunk overlap: zero;
- chunk-level (L_2) normalization;
- arithmetic mean pooling;
- final document-level (L_2) normalization.

5.10 Full-Corpus Production

The final embedding pipeline was applied to the complete analytical corpus.
The resulting database contains 2,249,255 document embeddings, corresponding one-to-one with the valid analytical articles.
Every vector has dimension 384, and no article in the final corpus has a missing document embedding.
The embeddings are stored at article level and constitute a semantic representation independent of the five predefined risk categories.
This distinction is essential for the next stage of the analysis. The thematic scores identify the degree to which an article is associated with a known risk category, whereas the document embedding describes the article's overall semantic content.

5.11 Role of Embeddings in the Empirical Strategy

The document embeddings are not themselves interpreted as emerging-risk indicators. Instead, they provide the semantic geometry required to construct such indicators.

In particular, the next chapter uses the vectors in two distinct ways: first, article-level novelty is constructed by comparing each article with semantically similar articles observed in the historical information set and second, semantic drift is constructed by aggregating embeddings within theme–month groups and measuring changes in the semantic representation of a risk category over time.

The embedding layer therefore connects unstructured news text to quantitative measures of semantic change:

$$\text{news text} \rightarrow E_i \rightarrow \{\text{semantic novelty, semantic drift}\} \tag{5.14}$$